Avançando com Typescript - Aula 22/08/2024

1. **Interfaces e Tipos Avançados:**
- Exploraremos o uso de interfaces para definir a forma de objetos em TypeScript.
- Introduziremos tipos avançados, como tipos de união, interseção e tipos condicionais.

2. **Classes e Herança:**
- Aprofundaremos o conhecimento sobre classes em TypeScript, incluindo herança e métodos estáticos.
- Demonstraremos como criar classes abstratas e implementar interfaces em classes.

3. **Módulos e Namespaces:**
- Aprenderemos sobre o sistema de módulos em TypeScript e como organizar nosso código usando importações e exportações.
- Introduziremos o conceito de namespaces para evitar conflitos de nome entre diferentes partes do código.

> 4. **Generics:**
> - Exploraremos o uso de generics em TypeScript para criar componentes reutilizáveis e flexíveis.
> - Veremos exemplos de como usar generics em funções, classes e interfaces.
>
> 5. **Decorators:**
> - Introduziremos decorators em TypeScript e como eles podem ser usados para adicionar metadados e funcionalidades extras a classes, métodos e propriedades.
>
> 6. **Configuração do TypeScript em Projetos:**
> - Aprenderemos como configurar um projeto TypeScript, incluindo a criação de um arquivo `tsconfig.json` e a instalação de bibliotecas de tipos.
>
> 7. **Manipulação de Erros:**
> - Veremos como lidar com o tratamento de erros em TypeScript, incluindo o uso de try/catch e lançamento de exceções personalizadas.

8. **Async/Await e Promises:**
   - Exploraremos como lidar com operações assíncronas em TypeScript usando async/await e Promises.
   - Aprenderemos como escrever funções assíncronas e lidar com erros em operações assíncronas.

---

1) Interfaces e Tipos Avançados:

Neste tópico, você explorará o uso de interfaces para definir a forma de objetos em TypeScript e aprenderá como trabalhar com tipos avançados, como união, interseção e tipos condicionais.

1.1. Interfaces

As interfaces em TypeScript servem para descrever a estrutura de um objeto. Elas definem quais propriedades e métodos um objeto deve ter, e também podem incluir propriedades opcionais e somente leitura.

```typescript
// Definição de interface
interface Pessoa {
    nome: string;
    idade: number;
    email?: string; // Propriedade opcional
    readonly cpf: string; // Propriedade somente leitura
}

// Exemplo de utilização
const pessoa: Pessoa = {
    nome: 'João',
    idade: 25,
    cpf: '123.456.789-00'
};

// Tentativa de alterar o CPF gerará um erro pois é somente leitura
// pessoa.cpf = '987.654.321-00'; // Erro: não pode alterar uma propriedade readonly
```

Explicação: A interface `Pessoa` define a estrutura do objeto. `email` é uma propriedade opcional e `cpf` é uma propriedade somente leitura, o que significa que não pode ser alterada após a inicialização.

1.2. Tipos de União

Os tipos de união permitem que uma variável **assuma** mais de um tipo possível. Isso é útil para restringir um conjunto de valores possíveis.

```
// Definição de tipos de união
type Status = 'ativo' | 'inativo' | 'pendente';

// Exemplo de utilização
const status: Status = 'ativo'; // Pode ser 'ativo', 'inativo', ou 'pendent
e'
```

Explicação: O tipo `Status` pode ser um dos três valores: `'ativo'`, `'inativo'` ou `'pendente'`. Isso garante que apenas um desses valores seja atribuído, evitando erros.

1.3. Tipos Condicionais

Os tipos condicionais permitem que um tipo seja definido com base em uma condição lógica. Eles são usados para criar tipos mais flexíveis e dinâmicos.

```
// Definição de tipos condicionais
type Cores = 'vermelho' | 'azul';
type Tamanho = 1 | 2 | 3;
type DescricaoCor<T> = T extends 'vermelho' ? 'Cor Quente' : 'Cor Fria';

// Exemplo de utilização
const cor: Cores = 'vermelho';
const tamanho: Tamanho = 2;

const descricaoCorVermelho: DescricaoCor<'vermelho'> = 'Cor Quente'; // Infe
rido como 'Cor Quente'
```

Explicação: `DescricaoCor` é um tipo condicional que verifica se `T` é `'vermelho'`. Se for, o tipo será `'Cor Quente'`, senão será `'Cor Fria'`. Isso adiciona lógica aos tipos e permite maior flexibilidade.

1.4. Tipos de Interseção

Os tipos de interseção permitem combinar vários tipos em um novo tipo que possui todas as propriedades dos tipos combinados. Isso é útil quando você precisa mesclar diferentes conjuntos de propriedades.

```
// Definição de interseção
type Jogador = {
```

```
    nome: string;
    idade: number;
    habilidades: string[];
};

type Gamer = {
    id: number;
    jogos: string[];
};

type JogadorGamer = Jogador & Gamer;

// Exemplo de utilização
const jogadorGamer: JogadorGamer = {
    nome: 'Maria',
    idade: 30,
    habilidades: ['futebol', 'tênis'],
    id: 123,
    jogos: ['FIFA', 'Mario Kart']
};
```

Explicação: `JogadorGamer` é uma interseção entre `Jogador` e `Gamer`, o que significa que esse tipo possui todas as propriedades de ambos. No exemplo, o objeto `jogadorGamer` possui as propriedades `nome`, `idade`, `habilidades`, `id`, e `jogos`.

Resumo da Aula

- **Interfaces**: Definem a forma dos objetos. Podem incluir propriedades obrigatórias, opcionais, somente leitura e métodos.
- **Tipos de União**: Permitem que uma variável tenha um conjunto específico de valores.
- **Tipos Condicionais**: Definem tipos com base em uma condição lógica, permitindo tipos dinâmicos.
- **Tipos de Interseção**: Combinam múltiplos tipos em um novo tipo que contém todas as propriedades dos tipos envolvidos.

Esses conceitos de TypeScript permitem criar aplicações mais seguras e robustas, já que o sistema de tipos ajuda a evitar erros comuns no código.

---

2) Classes e Herança

Classes e herança são pilares fundamentais da programação orientada a objetos, e em TypeScript, esses conceitos ganham mais força com a adição de tipagem estática.

2.1. Classes

Classes são moldes para criar objetos que encapsulam comportamento (métodos) e estado (propriedades). Com elas, é possível organizar o código de forma modular e reutilizável, facilitando o gerenciamento de objetos complexos.

```
// Definição de classe simples
class Pessoa {
    // O construtor inicializa as propriedades da classe
    constructor(public nome: string, public idade: number) {}

    // Método que retorna uma saudação
    saudacao(): string {
        return `Olá, meu nome é ${this.nome} e tenho ${this.idade} anos.`;
    }
}

// Instanciando um objeto da classe Pessoa
const pessoa = new Pessoa('João', 30);
console.log(pessoa.saudacao()); // Saída: Olá, meu nome é João e tenho 30 an
os.
```

Explicação: A classe `Pessoa` define duas propriedades: `nome` e `idade`. O construtor (`constructor`) é usado para inicializar essas propriedades ao criar uma nova instância da classe. O método `saudacao()` retorna uma mensagem personalizada baseada nas propriedades do objeto.

2.2. Herança

Herança é uma maneira de criar classes que reutilizam e estendem funcionalidades de outras classes. Uma subclasse (classe filha) herda todas as propriedades e métodos da superclasse (classe pai) e pode adicionar ou modificar seus próprios comportamentos.

```
// Definição de classe base (superclasse)
class Pessoa {
    constructor(public nome: string, public idade: number) {}

    saudacao(): string {
        return `Olá, meu nome é ${this.nome} e tenho ${this.idade} anos.`;
    }
}

// Definição de subclasse (herança)
```

```
class Estudante extends Pessoa {
    constructor(nome: string, idade: number, public curso: string) {
        super(nome, idade); // Chama o construtor da superclasse *
    }

    // Método adicional na subclasse
    apresentacao(): string {
        return `${super.saudacao()} Estudo ${this.curso}.`;
    }
}

// Instanciando e utilizando a subclasse
const estudante = new Estudante('Maria', 25, 'Análise e Desenvolvimento de Sistemas');
console.log(estudante.apresentacao());
// Saída: Olá, meu nome é Maria e tenho 25 anos. Estudo Análise e Desenvolvimento de Sistemas.
```

Explicação: A classe `Estudante` herda de `Pessoa` usando a palavra-chave `extends`. O método `apresentacao()` chama o método `saudacao()` da superclasse através da palavra-chave `super`. O construtor da subclasse também utiliza `super()` para inicializar as propriedades da classe pai antes de adicionar a nova propriedade `curso`.

> * Nesse caso, a **superclasse** é a classe `Pessoa`. Quando você usa `super(nome, idade);` dentro da subclasse `Estudante`, está chamando o **construtor** da classe `Pessoa` (a superclasse) para inicializar as propriedades `nome` e `idade`.
>
> A palavra-chave `super` faz referência à superclasse da qual a subclasse está herdando. No contexto do código:
>
> ```
> class Pessoa {
>     constructor(public nome: string, public idade: number) {}
> }
>
> class Estudante extends Pessoa {
>     constructor(nome: string, idade: number, public curso: string) {
>         super(nome, idade); // Chama o construtor da superclasse 'Pessoa'
>     }
>
>     apresentacao(): string {
>         return `${super.saudacao()} Estudo ${this.curso}.`;
>     }
> }
> ```

> Aqui:
>
> - `Pessoa` é a superclasse.
> - `Estudante` é a subclasse.
>
> Quando `super(nome, idade)` é chamado no construtor da classe `Estudante`, ele está chamando o construtor da classe `Pessoa` e passando `nome` e `idade` como parâmetros, de forma que a superclasse (Pessoa) inicialize corretamente suas propriedades.

2.3. Modificadores de Acesso

Em TypeScript, você pode usar modificadores de acesso como `public`, `private` e `protected` para controlar a visibilidade das propriedades e métodos dentro das classes.

```
class Funcionario extends Pessoa {
    private salario: number; // Propriedade privada, não acessível fora da classe

    constructor(nome: string, idade: number, salario: number) {
        super(nome, idade);
        this.salario = salario;
    }

    // Método público para acessar a propriedade privada
    getSalario(): number {
        return this.salario;
    }
}

const funcionario = new Funcionario('Carlos', 40, 5000);
console.log(funcionario.getSalario()); // Saída: 5000
// console.log(funcionario.salario); // Erro: 'salario' é privado e não pode ser acessado diretamente
```

Explicação: A propriedade `salario` é privada, o que significa que só pode ser acessada ou modificada dentro da classe `Funcionario`. O método `getSalario()` é público e permite acessar essa propriedade de fora da classe.

Resumo

- **Classes**: Estruturas que encapsulam comportamento e estado. Elas facilitam a criação de objetos com características definidas.
- **Herança**: Permite que classes compartilhem código entre si, criando hierarquias e promovendo a reutilização. As subclasses podem herdar ou sobrescrever métodos e

propriedades das superclasses.

- **Modificadores de Acesso**: Controlam a visibilidade de propriedades e métodos, garantindo encapsulamento e segurança.

---

3) Módulos e Namespaces:

Neste tópico, aprenderemos sobre o sistema de módulos em TypeScript e como organizar nosso código usando **importações** e **exportações**. Além disso, introduziremos o conceito de **namespaces** para evitar conflitos de nomes entre diferentes partes do código.

Explicação:

a) Módulos:

Os módulos em TypeScript são usados para **organizar** e **reutilizar código** em diferentes arquivos. Eles permitem dividir um programa em partes menores, facilitando a manutenção e a colaboração em projetos maiores. Cada arquivo TypeScript é tratado como um módulo separado por padrão, e o conteúdo de um módulo não vaza para o escopo global.

b) Importações e Exportações:

- **Exportações**: Permitem tornar definições de um módulo disponíveis para outros módulos. Podemos exportar variáveis, classes, interfaces e funções usando `export` ou `export default`.
- **Importações**: Permitem trazer definições de outros módulos para o escopo atual usando `import`. Há várias formas de importar, como `import { X } from 'módulo'`, `import * as X from 'módulo'`, ou `import X from 'módulo'`.

c) Namespaces:

Namespaces são usados para **organizar logicamente** um conjunto de definições em um escopo específico, ajudando a evitar conflitos de nome, especialmente em grandes projetos. Eles permitem agrupar classes, interfaces e funções dentro de um único "espaço de nome", similar a como objetos agrupam propriedades. Embora os módulos sejam mais amplamente utilizados hoje, os namespaces ainda são úteis em certos cenários.

Arquivo: `pessoa.ts`

```
export namespace PessoaNamespace {
    export class Pessoa {
        constructor(public nome: string, public idade: number) {}
```

```
        saudacao(): string {
            return `Olá, meu nome é ${this.nome} e tenho ${this.idade} anos.`;
        }
    }
}
```

Arquivo: `estudante.ts`

```
import { PessoaNamespace } from './pessoa'; // ou caminho de diretórios que você salvou esse Namespace

export namespace EstudanteNamespace {
    export class Estudante extends PessoaNamespace.Pessoa {
        constructor(nome: string, idade: number, public curso: string) {
            super(nome, idade);
        }

        apresentacao(): string {
            return `${this.saudacao()} Estudo ${this.curso}.`;
        }
    }
}
```

Arquivo: `principal.ts`

```
import { EstudanteNamespace } from './estudante';

const estudante = new EstudanteNamespace.Estudante('Maria', 20, 'Engenharia de Software');
console.log(estudante.apresentacao());
// Saída: Olá, meu nome é Maria e tenho 20 anos. Estudo Engenharia de Software.
```

Explicação:

- **Módulos**: Cada arquivo (`pessoa.ts`, `estudante.ts`, e `principal.ts`) é um módulo em TypeScript. Usamos **exportações** para tornar as classes disponíveis para outros arquivos e **importações** para acessá-las em arquivos diferentes.
- **Namespaces**: O código está organizado em namespaces (`PessoaNamespace` e `EstudanteNamespace`), o que permite agrupar logicamente as classes e evitar conflitos de

nomes. Cada namespace encapsula suas definições, como `Pessoa` e `Estudante`, prevenindo que nomes iguais se choquem.

- **Herança**: O `EstudanteNamespace` herda da classe `Pessoa` dentro do `PessoaNamespace`. Isso demonstra como namespaces e herança podem funcionar juntos, promovendo a organização e a reutilização de código.

> **Este exemplo é uma boa prática para projetos que precisam de grande organização e separação lógica do código, mantendo o código modular e prevenindo conflitos entre definições.**

EXERCÍCIOS

1) Exercícios de Interfaces e Tipos Avançados:

Exercício 1:

Objetivo: Trabalhar com interfaces e tipos de união.

- Defina uma interface chamada `Produto`, que contenha as propriedades `nome` (string), `preco` (number) e `categoria` (string).
- Defina um tipo de união chamado `FormaPagamento`, que pode ser `'dinheiro'`, `'cartão'` ou `'pix'`.
- Crie uma função que receba um objeto do tipo `Produto` e o tipo de pagamento, e retorne uma mensagem contendo as informações do produto e a forma de pagamento escolhida.

Exercício 2:

Objetivo: Utilizar interseção de tipos.

- Defina dois tipos: `Pessoa` e `Empregado`. O tipo `Pessoa` deve conter as propriedades `nome` (string) e `idade` (number). O tipo `Empregado` deve conter as propriedades `empresa` (string) e `salario` (number).
- Usando interseção de tipos, crie um tipo que combine `Pessoa` e `Empregado`.
- Crie uma função que receba um objeto desse tipo combinado e retorne uma mensagem que descreva o nome, idade, empresa e salário da pessoa.

---

2) Exercícios de Classes e Herança:

Exercício 1:

Objetivo: Criar uma classe e uma subclasse usando herança.

- Crie uma classe chamada `Funcionario`, com as propriedades `nome` (string), `cargo` (string) e `salario` (number).
- Adicione um método que retorne uma descrição do funcionário.
- Crie uma subclasse chamada `Gerente`, que herda de `Funcionario` e adiciona a propriedade `departamento` (string).
- Adicione um método na classe `Gerente` que retorne uma descrição detalhada, incluindo o departamento que o gerente supervisiona.

Exercício 2:
Objetivo: Trabalhar com herança e sobrescrita de métodos.

- Crie uma classe chamada `ContaBancaria` com as propriedades `titular` (string) e `saldo` (number).
- Adicione um método `exibirSaldo`, que imprime o saldo atual da conta.
- Crie uma subclasse chamada `ContaCorrente`, que herda de `ContaBancaria` e adiciona a propriedade `limiteCredito` (number).
- Sobrescreva o método `exibirSaldo` na subclasse `ContaCorrente` para incluir o saldo mais o limite de crédito disponível.

---

3) Exercícios de Módulos e Namespaces:

Exercício 1:
Objetivo: Organizar o código usando módulos.

- Crie um módulo chamado `Cliente.ts`, que exporta uma classe `Cliente` com as propriedades `nome` (string) e `email` (string).
- Crie um segundo módulo chamado `Pedido.ts`, que importará a classe `Cliente` e adicionará a classe `Pedido`, com as propriedades `cliente` (objeto do tipo `Cliente`), `produto` (string) e `valor` (number).
- Instancie objetos da classe `Pedido` e crie uma função que exiba o nome do cliente, o produto e o valor total do pedido.

Exercício 2:
Objetivo: Utilizar namespaces para organizar o código.

- Crie um namespace chamado `Financeiro`, que contenha duas funções: `calcularImposto` (recebe o valor e a taxa de imposto) e `calcularDesconto` (recebe o valor e a taxa de desconto).
- Dentro do namespace, adicione uma classe `Orcamento`, com as propriedades `valorTotal` (number) e `itens` (array de strings).
- Crie um arquivo separado onde você utilizará esse namespace, instanciando a classe `Orcamento` e utilizando as funções de cálculo de imposto e desconto para modificar o valor

total.

EXERCÍCIOS RESOLVIDOS

Na próxima aula, faremos a correção dos exercícios!